

# Homework 1

Computational Bootcamp, Summer 2026

Due: Wednesday, August 5

**Submission:** via Classroom 50 (link posted on course website). Clone your repository, and `git push` to submit. See the repository README for setup.

## Question 1: Present values

Write a function `present_value(payments, r)` that takes a vector of payments  $(y_1, \dots, y_T)$  and an interest rate  $r$  and returns

$$PV = \sum_{t=1}^T \frac{y_t}{(1+r)^{t-1}}$$

using a `for` loop. Check your function against the closed form for a constant payment stream ( $y_t = 1$  for all  $t$ ), which is a geometric sum.

## Question 2: Age-earnings profiles

A worker's log earnings over the life cycle are often approximated by a polynomial in experience  $x$ :

$$p(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n = \sum_{i=0}^n a_i x^i$$

Using `enumerate()` in your loop, write a function `p(x, coeff)` that computes the value of the polynomial given a point  $x$  and an array of coefficients `coeff`. Plot the implied earnings profile  $\exp(p(x))$  for  $x \in [0, 40]$  with coefficients  $(2.6, 0.08, -0.0015)$ .

## Question 3: Monte Carlo warm-up

Compute an approximation for  $\pi$  using Monte Carlo. You may only use `rand()` for random number generation. Hint: if  $U$  is a bivariate uniform random variable on the unit square  $(0, 1)^2$ , then the probability that  $U$  lies in a subset  $B$  of  $(0, 1)^2$  is equal to the area of  $B$ .

## Question 4: Simulating and estimating a Mincer regression

Let the data generating process for log wages be

$$\log(w) = \beta_1 educ + \beta_2 exp + \beta_3 exp^2 + \beta_0 + \sigma \varepsilon$$

where  $\varepsilon \sim N(0, 1)$  iid,  $educ$  and  $exp$  are scalars, and  $(\beta_0, \beta_1, \beta_2, \beta_3)$  are parameters.

- Draw  $N = 50$  values of  $educ \sim N(\mu = 14, \sigma^2 = 4)$  and  $exp \sim N(\mu = 15, \sigma^2 = 25)$  once, and hold them fixed.
- Using  $\beta_0 = 1.0$ ,  $\beta_1 = 0.08$ ,  $\beta_2 = 0.04$ ,  $\beta_3 = -0.0008$ , and  $\sigma = 0.35$ , draw an  $\varepsilon$  vector and generate  $\log(w)$ . Repeat until you have 200 simulated samples.
- Calculate ordinary least squares *manually* (i.e.  $\hat{\beta} = (X'X)^{-1}X'y$ ) for each of the 200 simulations. Plot histograms of the estimates of each parameter. Comment: are the estimators centered at the truth?

## Question 5: Firm survival

A firm's log productivity follows the AR(1) process

$$x_{t+1} = \alpha x_t + \sigma \varepsilon_{t+1}, \quad x_0 = 1,$$

where  $\{\varepsilon_t\}$  is iid standard normal. The firm *exits* the first time its productivity falls to zero or below: the exit time is  $T_0 = \min\{t \mid x_t \leq 0\}$  (set  $T_0 = t_{\max}$  if the firm survives the whole sample).

- With  $\sigma = 0.2$  and  $\alpha = 1.0$ , simulate 100 productivity paths (up to  $t_{\max} = 200$ ) and plot a histogram of exit times  $T_0$ .
- Plot the sample mean of  $T_0$  for  $\alpha \in \{0.8, 1.0, 1.2\}$ . Also report, for each  $\alpha$ , the fraction of firms that never exit (i.e. that hit  $t_{\max}$ ). Interpret: what does  $\alpha$  do to firm survival, and how much can you trust the sample mean of  $T_0$  when a large share of firms is censored at  $t_{\max}$ ?

## Question 6: Newton's method

Recall that the root of a univariate function  $f(\cdot)$  is an  $x$  such that  $f(x) = 0$ . Starting from a guess  $x_0$ , Newton's method repeats

$$x^{n+1} = x^n - \frac{f(x^n)}{f'(x^n)}$$

until  $|x^{n+1} - x^n|$  is below some tolerance threshold. Code a function that implements Newton's method, accepting a function `f`, its derivative `f_prime`, a starting guess `x_0`, a tolerance `tol`, and a maximum number of iterations `maxiter`. Test it with  $f(x) = (x - 1)^3$  and with a nontrivial economic example: find the interest rate  $r$  that prices a 10-year bond paying a coupon of 5 per year with face value 100 at an observed price of 95, i.e. solve

$$f(r) = \sum_{t=1}^{10} \frac{5}{(1+r)^t} + \frac{100}{(1+r)^{10}} - 95 = 0.$$

(You may compute  $f'$  analytically.)

## Question 7: Make it fast

The file `slow_growth.jl` in your homework repository solves the deterministic optimal investment problem from Lecture 2,

$$V(K) = \max_{K'} \{ \log(c) + \beta V(K') \}, \quad c = K^\theta + (1 - \delta)K - K',$$

with  $\theta = 0.36$ ,  $\delta = 0.025$ ,  $\beta = 0.99$ , and a capital grid from 0.01 to 45 with 1000 points — but it is written *terribly*: global variables, no functions, type instability, unnecessary allocations.

Rewrite it using the practices from Lecture 2 (a `@kwdef` struct for primitives, functions for the Bellman operator and solver, pre-allocation). Use `BenchmarkTools.jl` to report the speedup you achieve, and list which changes bought you the most.